

Exchange-based Incentive Mechanisms for Peer-to-Peer File Sharing

Kostas G. Anagnostakis and Michael B. Greenwald
 CIS Department, University of Pennsylvania
 200 S. 33rd Street, Philadelphia PA 19104, USA
 {anagnost,mbgreen}@dsl.cis.upenn.edu

Abstract—Performance of peer-to-peer resource sharing networks depends upon the level of cooperation of the participants. To date, cash-based systems have seemed too complex, while lighter-weight credit mechanisms have not provided strong incentives.

We propose exchange-based mechanisms for providing incentives for cooperation in peer-to-peer file sharing networks. Peers give higher service priority to requests from peers that can provide a simultaneous and symmetric service in return. We generalize this approach to n -way exchanges among rings of peers and present a search algorithm for locating such rings. We have used simulation to analyze the effect of exchanges on performance. Our results show that exchange-based mechanisms can provide strong incentives for sharing, offering significant improvements in service times for sharing users compared to free-riders, without the problems and complexity of cash- or credit-based systems.

I. INTRODUCTION

Peer-to-peer systems provide a powerful infrastructure for large-scale distributed computing applications, mainly because of the wide-spread cooperative resource sharing among participants. Cooperation and the existence of a critical mass of participants with sufficient resources are key elements for enabling a variety of novel applications such as file sharing, large-scale content distribution, and distributed data processing. Performance in such systems depends on the level of cooperation by the system's participants. While most existing peer-to-peer architectures have assumed that participants are generally cooperative, there is growing evidence from widely deployed systems suggesting the opposite. For instance, one study of the Gnutella file sharing system shows that almost 70% of the peers only consume resources and do not share any files with the network[1]. The result of this kind of non-cooperation can vary between tolerable service degradation and complete system collapse depending on design goals and performance requirements.

Such problems have recently motivated work on incentive mechanisms for peer-to-peer systems that stimulate cooperation between self-interested participants. Systems such as KaZaA[2] attempted some rather naive methods where each peer announces its "participation level", computed locally as a function of uptime, download and upload volume, and give priority to remote peers that claim high participation

levels. However, this is easily subverted since peers can claim anything with a simple modification to their software. In fact, such hacks are easily accessible [3] and widely used[4]. Other proposals to date require the use of a credit system which can be either centralized or decentralized. Centralized mechanisms[5], [6] (e.g., using micropayments issued by a trusted server or a centralized transaction clearing center) inherit the typical disadvantages of centralized designs (such as indexing) in that they introduce a single point of failure, may put a significant burden on a single peer and, perhaps most importantly, it may be hard to design the right incentives for one or more peers to take up such a demanding and sensitive role. Recent proposals for decentralized credit mechanisms[7], [8] are based on distributed hash tables (DHTs) [9], [10], [11] and therefore inherit another set of problems. For instance, heterogeneous node capabilities make efficient allocation decisions hard, transient peer participation may significantly stress reconfiguration performance, and there are known classes of attacks that are likely to be directed against the credit system service given its importance[12].

As an alternative, we propose a more lightweight approach that avoids the complexities of credit mechanisms. Rather than building a system based on principles of monetary or credit economies, we structure the system as a more primitive *exchange* or *barter* economy. Users directly trade resources between themselves, so little or no long-term bookkeeping is required. Requests from peers that can provide a simultaneous, symmetric, service in return (*exchange transfers*) are given higher priority. The service need not be directly to the provider (a *pairwise exchange*), but more generally priority is given to peers who participate in N -way *exchanges* to which the provider currently belongs. N -way exchanges are implemented as *rings* of N peers, where each peer is served by its predecessor and serves its successor in the ring. Non-exchange transfers are only served if no other exchange is possible and peers have spare capacity. The preference given to exchange transfers provides a strong incentive for participants to cooperate.

The rest of the paper is organized as follows. In Section II we describe prior work on incentive mechanisms in peer-to-peer systems, and discuss the complexities of credit-based systems and the limited effectiveness of the incentives in lighter-weight systems. In Section III we present the proposed exchange mechanisms and discuss several key design issues with respect to efficiency and security. In Section IV we

present a simulation study analyzing the exchange mechanisms and their effect on file sharing system performance. In Section V we discuss open issues and directions for extending the results presented here. We summarize and conclude in Section VI.

II. RELATED WORK

The first known use of payment-based incentive mechanisms in peer-to-peer file sharing was in the now defunct MojoNation network[5]. Each user was given an initial endowment called Mojo which he could spend on purchasing files from other peers. The main limitations of this approach is that all transactions had to be cleared in a centralized system, and users were burdened with managing their Mojo.

A distributed cash-based system for peer-to-peer systems is presented in [7]. The system uses a currency called *karma* which is maintained for each user by a collection of random participants called a *bank-set* that is located using a DHT lookup. Users need to negotiate the price of serving an object through an auction mechanism, and coordinate with the bank-set for transferring karma between accounts. Each user receives an initial amount of karma when signing up with the system, and the system ensures that the rate at which users can create new identities is limited through the use of a cryptographic puzzle. To address inflation or deflation, the system needs to periodically normalize the total amount of currency in the system. The result is a fully-fledged economic system that is in principle more flexible than an exchange system. In an idealized setting, a cash-based scheme may be able to offer a stronger performance advantage to contributing peers, as it is not subject to the “*double coincidence of wants*” constraint that drives exchanges. In practice however, the cash-based approach has two main limitations.

First, it suffers from all the complexities of currency management. If the mechanisms used to negotiate prices, adjust accounts to inflation and deflation and manage a user’s budget are not made completely transparent to the user, then such a system is likely to have a high cost in terms of *user attention*[13] which is suggested as a major reason why micropayment schemes are unlikely to get wider acceptance in general[14]. The feasibility of such mechanisms has not been proven to date.

Second, the need to provide start-up funds to new users creates a potential loophole in the economy. Specifically, the cryptographic puzzle used for protecting against the creation of new user identifiers and transfer of credit to existing active users may not be sufficient, as it may not be able to offer a satisfactory trade-off between keeping fake accounts out and allowing legitimate new users in. In essence, it is possible to earn cash in return for CPU cycles, without doing any useful work for the system.

A lightweight, *pair-wise* credit system is implemented in the eMule system[15]. The goal of the credit system is to reward users contributing to the network by reducing their waiting time in the upload queue. For each request in the upload queue the peer computes the *Queue Rank* based on a scoring function that depends on the current waiting time

for the request, as well the upload and download volumes for the peer. The main advantage of this scheme is simplicity: there is no communication overhead and a peer only needs to maintain the upload and download volumes for each peer it has communicated. The approach is cheat-proof in the sense that peers have no reason to tamper with the credit file. However, anecdotal evidence[16] suggests that the approach does not consistently provide a clear performance advantage to users who contribute resources to the network. Although there is no clear evidence in terms of measurements to determine precisely why this is happening, the credit approach appears to have two main limitations.

First, it is hard for a peer to strategize in terms of what peers he wants to earn credit from in order to maximize expected benefits. A large fraction of peers may be disconnected resulting in delays in rewarding credit; other peers may leave the system permanently, resulting in loss of credit; others may not have any object the peer is interested in, and some may not share content at all. The use of “waiting time” as a factor in computing queue rank further complicates this problem. It results in giving weaker performance advantage to users with established credit, as peers that do not have any credit can still use the system if they are patient enough. Tuning the scoring function to reduce the effect of waiting time is possible, but result in never serving users that don’t have established credit, even if establishing credit with those peers could be beneficial in the future.

One practical workaround to address this problem¹ is to control the set of shared files in a way that increases credit with peers likely to be useful for a given set of requests in the near future. For instance, if a peer is requesting an object in category *C*, then it makes sense to limit sharing to only those objects that are already available and belong to category *C*. Assuming that remote peers sharing the requested object are likely to request objects from the same category, the peer is more likely to earn credit and therefore improve queue rank and reduce waiting time on those peers. In this scenario, the credit system essentially *approximates* exchanges, at the cost of additional effort to get the conditions right for this to happen.

A second problem with the pairwise credit system is that there is no clear incentive for individual peers to cooperate in supporting the credit system, although this approach could in principle lead to a better global operating point. There is also no strong individual incentive not to honor credit, but in practice certain variants of the eMule client do not support the credit system, which also means that a fraction of the credit earned essentially gets lost. The mechanism does not directly penalize clients for this type of defection, and building additional protection (e.g., monitoring compliance and maintaining blacklists) adds complexity.

In [17] the authors argue that peer-to-peer “bartering” is an appropriate way to bootstrap peer-to-peer economies, focusing on systems like PlanetLab[18] where peers share basic resources like computing, storage and network capacity.

¹This has been suggested on message boards as a strategy that has worked in practice.

They propose the exchange of signed resource tickets between system participants that can be stored, traded and used for allocating resources. Strictly speaking, this is closer in spirit to credit economies involving personal debt certificates than barter. This approach is well suited for systems with a small set of homogeneous resources like CPU, storage and network capacity. In such systems the exchange mechanisms are unlikely to be useful, as there is little meaning in instant exchanges of same-type resources. In contrast, file sharing systems are content-oriented, providing a high level of specialization in terms of the objects served by each peer. This fits well with the instant exchange model.

The work most closely related to ours is Bittorrent, a system for large-scale content distribution where peers exchange blocks of the same file in an effort to expedite the distribution of large files [19]. The approach is more limited in that it only supports pairwise exchanges on the same file, and appears to be vulnerable to freeriding middlemen (we defer discussion of this flaw and solutions to Section III-B). No evidence is provided on the actual contribution of the exchange mechanism, as the system was not used at a scale comparable to popular file-sharing systems to observe any substantial diversity in peer behavior. To the best of our knowledge, our study is the first to investigate the effect of exchange mechanisms on peer performance and their value as an incentive mechanism in a file-sharing system.

III. EXCHANGE MECHANISMS

In this paper we consider a file sharing system where each peer has fixed upload and download capacity. The upload capacity is more likely to be the resource bottleneck than the download capacity. To manage the upload link, we respond to all requests in relatively large, equal, fixed-size, blocks. We assume that the system supports partial transfers and that peers can download different parts of the same object concurrently from multiple sources. To focus on the main point of this paper, we ignore the details of object lookup. We note that our approach can work with several known search mechanisms including broadcast in Gnutella-like networks or a DHT query in systems like Chord. When a peer is interested in an object it can use one of these methods to locate up to a certain fraction of peers that currently have the object.

Each peer has an *incoming request queue* (IRQ) where remote peers register their interest for a local file. A transfer to satisfy a request is initiated if two conditions are met. First, there must be sufficient capacity at both peers for the transfer. The local peer must have upload capacity (an open fixed-size slot on the upload link), and the remote peer must have sufficient download capacity. Second, either the transfer is an *exchange transfer*, or else no other request in the IRQ is both an exchange transfer *and* satisfies the first condition.

In practice, the local node does not check the download capacity of the remote node, but assumes it is sufficient. Inadequate download capacity terminates the transfer when the remote node cannot receive its incoming request, it terminates its outgoing upload, and issues the request again when a download is feasible.

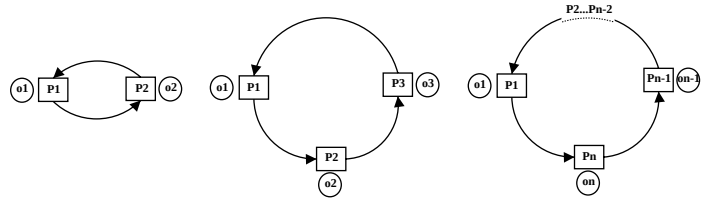


Fig. 1. Pairwise, 3-way and n-way exchanges

All exchanges are performed one fixed-size block at a time. Transfers are terminated if one of the two communicating peers disconnects, if the transfer is completed, or if the source deletes the object. It is quite common for one side to terminate first, when it completes its own download, because object sizes may differ and the system allows partial and concurrent transfers.

Non-exchange transfers will only be served if no exchange is possible and the peer has a free upload slot, although these slots will be reclaimed as soon as another exchange becomes possible. Peers who share more are more likely to be able to participate in an exchange, directly rewarding them with faster transfers. Thus, the power of the proposed approach is derived from the priority given by the system to exchange over non-exchange transfers.

A. Exchange transfers

Peers must give priority to exchange transfers. It is therefore imperative that feasible exchanges be identified.

Pairwise exchanges are easily detected. Each peer A regularly examines its incoming request queue and determines if, for any pending request, the remote peer B has some object that A is interested in that would qualify for a pairwise exchange. Although pairwise exchanges are simple, unfortunately, requests frequently do not resolve into convenient pairs.

Fortunately, it is easy to compute feasible N -way exchanges. Let G be the directed graph whose vertices are nodes in the peer-to-peer system, and whose labeled edges represent requests. An edge from node p_1 to p_2 with label o_e represents a request from p_1 to p_2 for object o_e . It is clear that any cycle of length n in G represents a feasible n -way exchange.

How can we compute cycles in G , a potentially enormous graph? First, we have empirically determined that n -way exchanges, where $n > 5$, do not substantially improve the likelihood of successful exchanges over exchanges where $n = 5$ (see Section IV). Therefore, it is sufficient to limit the search for cycles to chains of up to 5 predecessors. Second, we note that a request from A in the incoming request queue of B represents an edge in G from A to B , and therefore peers already have information about a partial local subgraph of G .

Each peer maintains a *request tree* as follows. A peer with no incoming requests has an empty Request Tree. For peers with non-empty incoming request queues, let each request in the IRQ include the contents of its request tree (pruned to a depth of 5). A 's Request Tree consists of an implicit root, A , as the parent of the set of Request Trees accompanying each entry in the IRQ. Then, A can initiate an n -way exchange if any peer in the Request Tree owns any object currently desired

by A . If a suitable peer is found, and that peer appears at depth n in the tree (the depth includes A as the root), then we can construct a ring of n peers, P_i for $0 \leq i < n$, each carrying object o_i and requesting object $o_{(i+1) \bmod n}$. Each peer provides an object to their predecessor and gets an object from their successor.

A inspects the Request Tree before transmitting any request and after receiving each request. Prior to transmission of a request for object o_e , A inspects the entire Request Tree to see if any peer provides o_e . On receipt of each request, r , A need only inspect the incoming Request Tree associated with r — but it checks the peers in the Request Tree for *any* object that A still wants.

Note that at the time A decides to request object o_e it “discovers” a (possibly incomplete) set of peers who provide object o_e , but it actually issues requests to only a subset of those peers. It can use the original provider list to compute a cycle containing a peer, p_i , even if it did not originally transmit a request to p_i . At the initial request time, A had no preference for p_i because A had no way of knowing that p_i was a potential participant in an n -way exchange.

In practice, A must circulate a token through the proposed ring to determine whether everyone is still willing to serve. The ring can be invalid for several reasons. First, in the time between the original requests and the ring initiation attempt, some peers may have gone offline, or crashed. Second, other peers may have already constructed rings of their own, including some of A ’s intended participants (it is possible that several peers along the intended cycle will attempt to create the same ring roughly simultaneously).

An interesting question is how to prioritize different feasible exchanges that can satisfy a given request. In principle, a preference for larger rings should have a positive effect on overall performance, as more peers are served. On the other hand, peers would prefer smaller rings as the search cost is lower, and the expected exchange volume is also more likely to be higher for smaller rings, as the probability of a peer either disconnecting or completing is higher for larger rings. Assuming peers care less about global performance and more about their own benefit, there is no clear incentive to put additional effort into looking for larger rings when even a pairwise exchange has been located that has equal (or higher) expected utility.

B. Preventing cheating

Since the system gives priority to exchange transfers, malicious peers may attempt to cheat. For example, a peer could claim that there is an exchangeable object available and serve junk in exchange for real data.

Several mechanisms can be used to address this problem. Peers can locally blacklist cheating peers and refuse to serve them later. In a large and dynamic system this is likely to be ineffective as cheaters may perform well enough even if they can cheat each peer only once. Cooperative blacklisting could help tackle this problem, although it requires additional mechanisms which may themselves be subject to attacks. In both cases, the problem persists if it is easy for a peer to

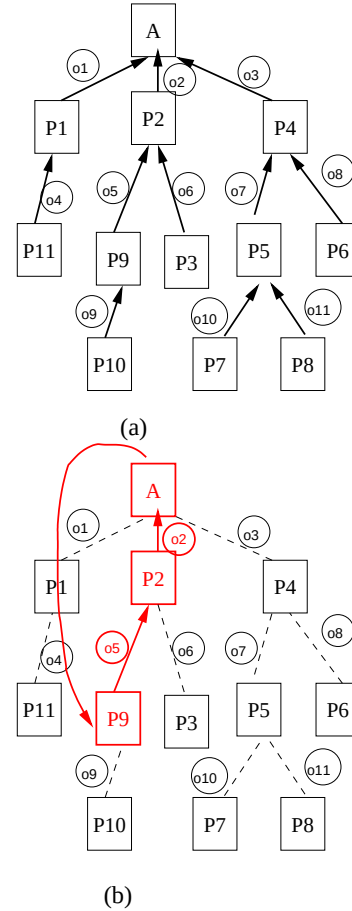


Fig. 2. A can be served by some object on $P9$. The entire request tree is shown in (a). The cycle for the 3-way exchange that A tries to initiate is shown in (b). $P3$ may have simultaneously discovered a cycle through a target of $P2$ other than A . Both rings need $P2$, so only one will be initiated successfully.

assume a new identity that is not blacklisted, as is the case in most file sharing systems today[20].

It is possible to limit the damage done by cheating by exchanging blocks synchronously and validating each received block before transferring the next one. This requires a trustworthy source of information for the actual valid checksums of the blocks being probed. The maximum benefit for a cheater in this case would be equal to the block size. If the block size is $b_{exchange}$ bytes and the round-trip time between the two peers is t_{rtt} seconds, this limits the maximum exchange rate to $b_{exchange}/t_{rtt}$ bytes/second. As this may be less than the slot capacity, peers may want to use a window protocol and increase the window size to fill up the slot capacity-delay product, at the expense of increasing risk. A reasonable approach would be to start the exchange with a small window and increase after a number of rounds. A cheater would need to have at least a few real blocks in order to increase the window. It is very likely that even this level of cooperation would have a positive effect on the system as a whole.

Another problem is that a peer could act as a middleman between two peers that could perform an exchange directly with each other, and obtain an object without doing any useful work for the system. Specifically, let's assume that peer A has

object x and wants object y , and peer B has object y and wants object x . The cheating peer C , interested in object x claims that he has object y and wants object x when talking to A , and that he has object x and wants object y when talking to B . Peer C would start getting blocks of y from B and exchanging them for blocks of x with A which in turn are passed to B for more blocks of y . In this scenario, peer C does not contribute any useful work to the system, and can still get high-priority service. If this is allowed to happen, then the exchange-based incentives break down.

Tighter control is needed to address this problem, involving the use of a trusted peer as a mediator. Both directions of the transfer can be encrypted, each with a secret key only known to the sending peer and the mediator. In the control header of each transfer block, the sending peer also includes a *peer-of-origin* identifier. The control header is also encrypted, so that a middleman cannot modify it. When the transfer is completed, the trusted peer mediates the exchange of the secret keys, after ensuring that neither side of the exchange has cheated. The mediator can do this by verifying the validity of a certain small number of randomly chosen blocks from each side of the transfer. The keys are sent to the peers indicated in the control header of the test blocks. In this way, a middleman would not be able to decrypt the blocks he peddled between the two peers in the scenario discussed above, and his participation in the transfer would offer him no benefit.

One remaining issue with this approach is that the middleman can initially obtain two blocks, one for each object peers A and B are interested in, and carry out small, one block transfers with each peer, and then presenting the newly acquired block for an exchange with the other peer. Since he starts this process with real data that is not encrypted, the protection offered by the mediator is not sufficient in this case. Although we do not have a similar solution for this problem, we argue that this way of increasing one's performance without doing useful work is unlikely to be possible at a large enough scale to be practical for cheaters as a general strategy, and a threat to the exchange-based system. First, the cheating peer needs to wait in low-priority queues to get the 'bait' blocks anyway, for both files, adding some latency to the process. Second, the number of potential "victim" peers decreases with the number of blocks the cheater has available. Third, since the cheater needs to have two blocks, one for each peer, he is also constrained by the number of peer-pairs interested in those blocks. Fourth, the cheater is wasting his resources because he is using part of his upload capacity for an object that is totally useless to him. unless of course he is interested in both objects. if not, he may be better off using this capacity for real exchanges. Fifth, the peers he is targeting are likely to be talking to each other already so they may be uninterested in what he has to offer, and they may have already committed all of their upload capacity to each other. Finally, additional constraints can be designed into the system to discourage this behavior, such as giving higher priority to longer exchanges.

Since users are considered to be *self-interested* rather than malicious, the best way to discourage this behavior is to offer an alternative that gives them better performance at a lower

peer	upload	has	wants
A	10	-	x
B	5	x	y
C	10	y	x
D	10	y	x

TABLE I

EXAMPLE MIDDLEMAN SCENARIO RESULTING IN NON-RING EXCHANGE

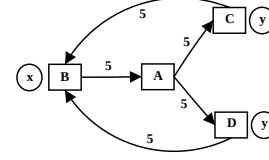


Fig. 3. Example of non-ring exchange

cost, is useful for the system as a whole, and respects their desire not to store or share objects. For instance, consider the scenario of Table I.

Although peer A has no exchangeable object, it is possible to substitute a pure object exchange with a mixed object-capacity exchange as shown in Figure 3: peer B sends x to A (5 upload units), peer A forwards x to C and D (5 upload units each for a total of 10), and C and D send y to A (5 upload units each for a total of 10). In this scenario, the result is the same for C and D compared to a pure object exchange, but both A and B increase their utility, since B gets object y at a rate of 10 when he would normally only be able to get it at a rate of 5, and peer A gets object x at a rate of 5 when he would not be able to participate at all in a pure object exchange. Of course, this requires a generalization of the exchange mechanism to non-ring topologies, which we do not discuss or analyze further in this paper.

IV. SIMULATION

A. Environment

We simulate a small, 200-node file-sharing system where each peer has fixed and asymmetric upload and download capacity e.g. the available capacity is not affected by other user traffic and there is typically much more download than upload capacity. We assume that the core network is sufficiently overprovisioned, delay and loss are negligible so that the only bottleneck in the system is a peer's connection.

The object popularity model is similar to the model presented in [21]. Objects are organized in categories. Each peer is interested in m categories, which are selected at initialization time. The popularity of a category of rank i is computed as $F_c^i = 1/(1 + i \cdot f_c)$ i.e. the probability of a request for an object in category i is $p_c^i = F_c^i / \sum_{j=0}^m F_c^j$. For each of the m categories assigned to each peer we also assign a local preference distribution with uniformly random weights for each category. The local preference distribution is independent from global popularity. When a peer issues a request, it chooses a category based on the local preference distribution, and then picks an object in that category, also based on a distribution where the popularity of an object of rank i is

number of peers	200
download capacity	800 kbit/s
upload capacity	80 kbit/s
ul/dl slot size	10 kbit/s
content categories	300
objects per category	uniform(1,300)
categories/peer	uniform(1,8)
category popularity	$f=0.2$
object popularity	$f=0.2$
object size	20 MB (all objects)
storage capacity per peer (nr. of objects)	uniform(5,40)
queue for incoming requests	1000
max pending objects	6
fraction of freeloaders in system	50%

TABLE II
BASIC SIMULATION PARAMETERS

computed as $F_o^i = 1/(1+i \cdot f_o)$ and the probability of a request for an object in category i is $p_o^i = F_o^i / \sum_{j=0}^m F_o^j$. For $f = 0$ the distribution becomes uniform, and for $f = 1$ it becomes zipf-like. Note that measurements of real-world file-sharing systems suggest zipf-like locality[22].

Each peer has up to a maximum number of pending requests. Requests are generated fast enough so that each peer reaches this maximum early enough in the simulation, and throughout an experiment a new request is issued as soon as a pending download is completed. As the factor f of the object popularity distribution increases, peers are increasingly likely to request an object already available locally. In reality, peers are unlikely to request objects they already have and the effect of such “cache hits” on our measurements could be misleading. To avoid such effects we therefore choose to ignore hits and continue to generate candidate requests until a miss is found. This may shift the distribution of requests more towards uniform, but the resulting bias is more on the conservative side.

Each peer can store up to a maximum number of objects. We initially place objects on each peer based on the peer’s category preferences. In regular intervals, peers examine their storage and remove random objects if the maximum number of objects is exceeded. A peer postpones removing an object if it is used in an ongoing exchange.

The system parameters for simulation are shown in Table II.

B. Results

The key metric for peer performance in file sharing systems is object download time. We therefore obtain the mean object download time for sharing and non-sharing users in a non-exchange, pairwise, 5-2-way (e.g. choosing longer over shorter exchange rings), and 2-5-way (e.g. choosing shorter over longer rings) exchange system. We first look at behavior as load in the system increases. The results are shown in Figure 4. As expected, as the upload capacity is reduced, the mean download time increases for both sharing and non-sharing users, but increases faster for non-sharing compared to sharing users. This happens because as the system gets more loaded,

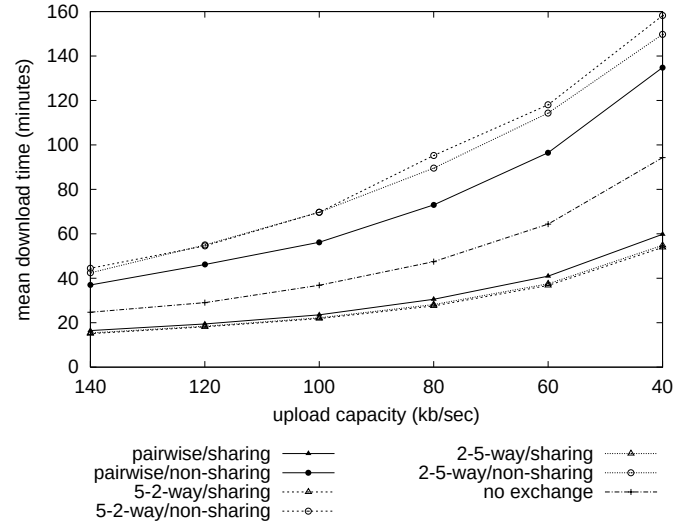


Fig. 4. Mean download time vs. upload capacity

resources are shifted to sharing users because we prioritize exchanges over non-exchanges and so a larger fraction of the upload resources can be given to users that can participate in exchanges. For 40 kbit/s upload capacity the use of pairwise exchanges results in download times for sharing users that are less than half of the download times for non-sharing users. The use of higher-order exchanges in addition to pairwise (denoted as 2-5-way and 5-2-way in the graph) gives sharing users four times better performance than non-sharing users. When using the exchange mechanism the improvement for sharing users is also significant compared to a system where no exchange mechanisms are introduced (“no exchange” in the graph): downloads are roughly twice as fast when exchanges are used. This observation suggests that sharing peers have a good incentive to deploy the proposed exchange mechanism.

In Figure 5 we present the fraction of exchange requests in the system as load increases. We see that the fraction of exchange requests increases almost linearly with load; as the object popularity model does not change, the difference is because as load increases a larger fraction of the transfer slots on each peer are given to exchange transfers. We also see that pairwise only performs slightly worse than 5-2-way and 2-5-way. If peers aggressively seek out feasible longer exchange rings before resorting to shorter rings, the system performs slightly better than if peers only look for longer rings when no shorter rings are feasible.

The benefit of seeking and using higher-order exchange rings is shown in Figure 6. We observe that there is a significant difference between $N = 5$ and $N = 2$, suggesting that higher-order exchanges are indeed valuable. However, much larger rings ($N > 5$) do not offer any substantial improvement.

In Figure 7 we present the distribution of the amount of data transferred per session. We see that exchanges have a higher transfer volume, as normal transfer sessions tend to be canceled and replaced by exchanges. We also observe that the transfer volume is much higher for shorter rings than for longer, as there is a higher probability of a peer completing

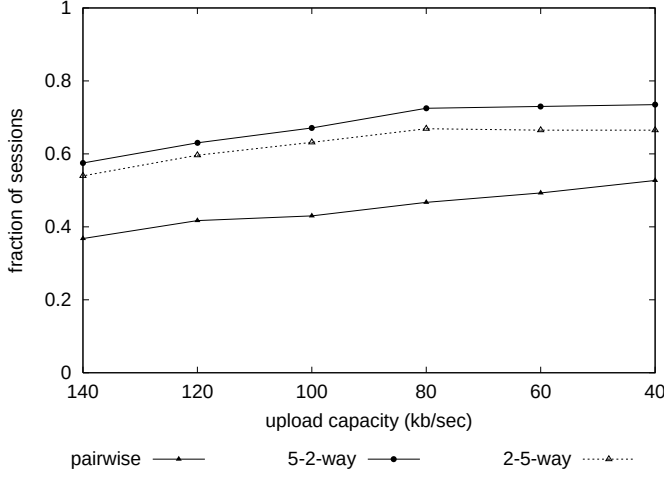


Fig. 5. Fraction of exchange transfers vs. upload capacity

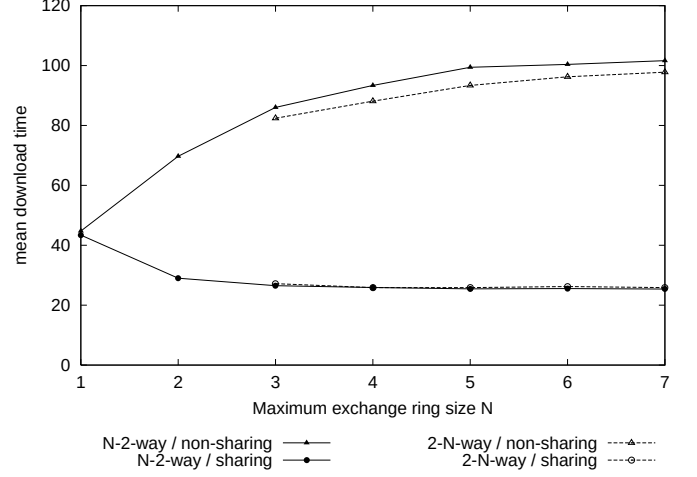


Fig. 6. Differentiation of mean download times for sharing vs. non-sharing users as a function of the maximum exchange size

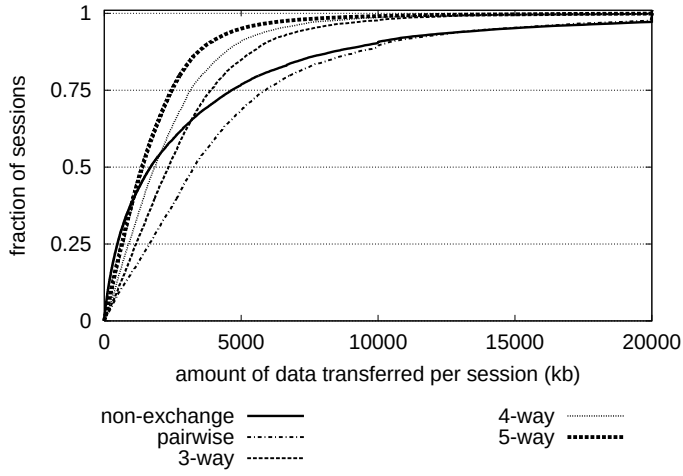


Fig. 7. CDF for transfer bytes per traffic type

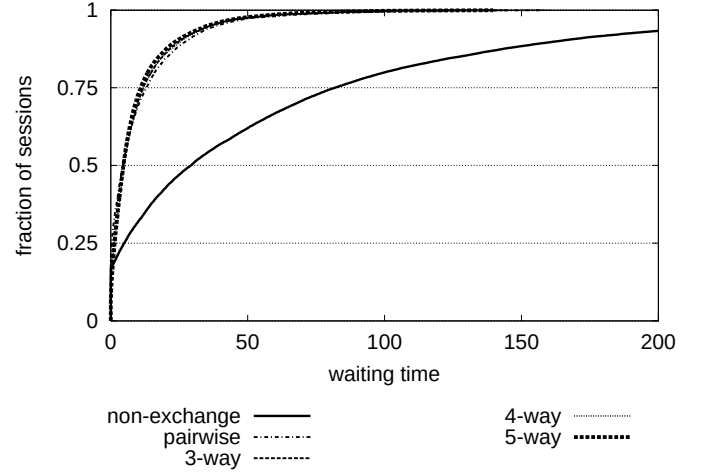


Fig. 8. CDF for transfer starting times per traffic type

a transfer and therefore dropping the exchange when there are many peers compared to when there are only two peers. This helps explain why higher-order exchanges contribute less than pairwise exchanges to the overall improvement. Note that in the current simulation model peers always pick the first feasible exchange in the search process. There may be other feasible exchanges with a longer expected life-cycle. Thus, the system could be modified for determining the best possible exchange, at the expense of increased search time and cost.

In Figure 8 we present the distribution of the waiting times for different classes of transfers. The waiting time for a session is the difference in time between the original object request and the start of a transfer. We see that waiting times for non-exchange transfers are substantially worse than for exchange transfers, as the system gives absolute priority to exchanges. This difference is the key reason why exchanges provide significantly better performance to sharing users. The waiting time is only slightly worse for higher-order exchanges compared to pairwise exchanges, meaning that this is not the cause for the relatively smaller benefit of higher-order

exchanges.

We also determined the effect of the object popularity distribution on performance. In Figure 9 we show the mean download time for different types of exchange configurations as a function of the object and category popularity factor f . As expected, the difference in performance between sharing and non-sharing users increases as the factor f approaches 1, resembling a zipf-like distribution, although the relative benefit is significant even when object popularity is more evenly distributed. In this experiment, the difference between 5-2-way and 2-5-way exchanges becomes more clear, and it seems that 2-5-way perform slightly better, not because they improve the performance of sharing users but because they reduce performance of non-sharing users. This happens because exchange transfers displace non-exchange transfers and 2-5-way are longer-lived on average than 5-2-way, even if they have similar aggregate transfer volumes, as shown in Figure 10. Because they have similar transfer volumes they do not affect the performance of sharing users as much, but, as they are more long-lived, they tend to displace non-exchange

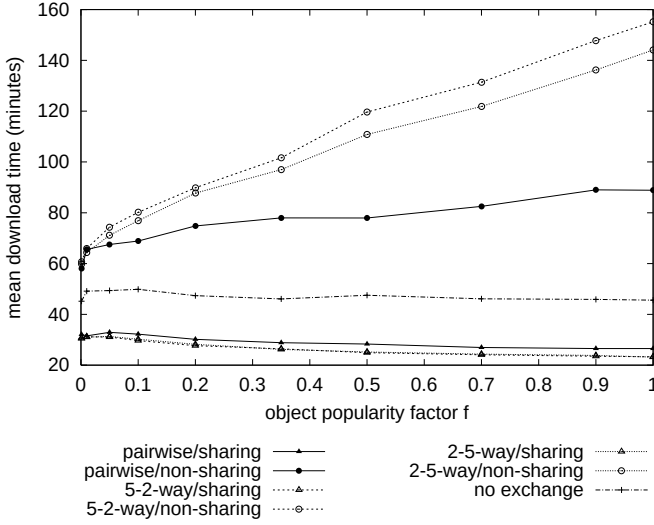


Fig. 9. Mean download time vs. object popularity factor

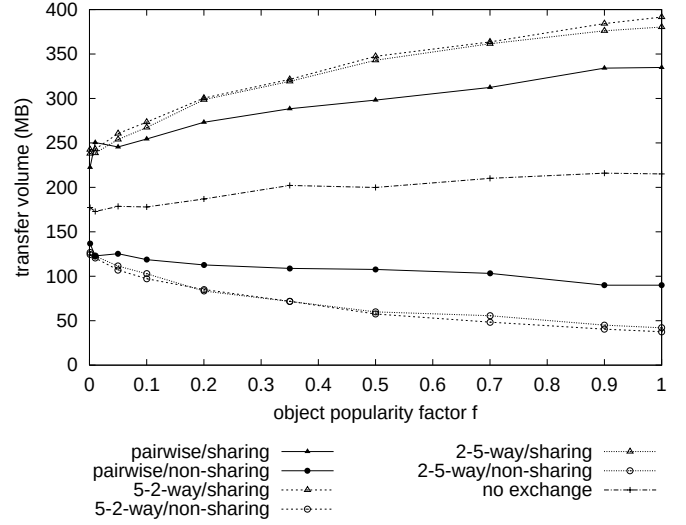


Fig. 10. Transfer volume (MB) vs. object popularity factor

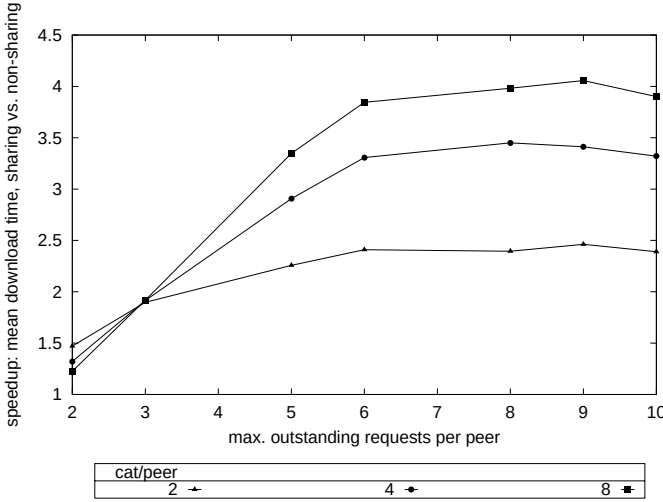


Fig. 11. Ratio of mean download times for different maximum pending request sizes and number of categories each peer is interested in

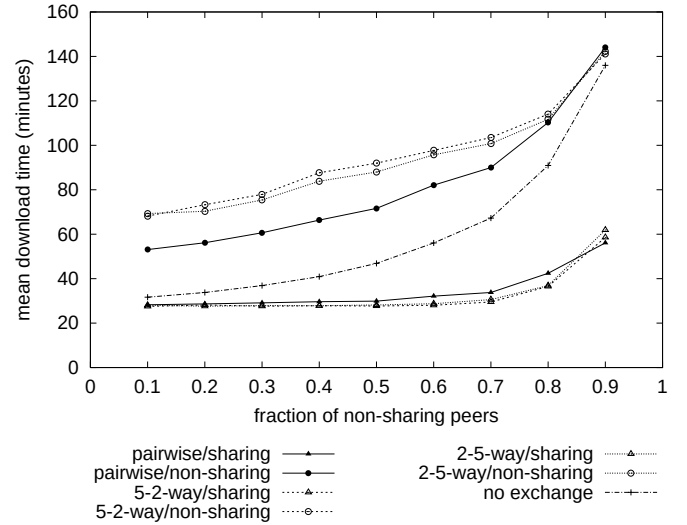


Fig. 12. Mean download times vs. fraction of non-sharing peers in the system

transfers for a longer time. These results indicate that giving preference to pairwise over higher-order exchanges is a good engineering choice, in addition to being cheaper in terms of search cost.

In Figure 11 we present the ratio of mean download times between sharing and non-sharing users as a function of the maximum number of outstanding requests on each peer as well as the number of categories each peer is interested in. The maximum number of outstanding requests increases system load, but also increases the number of feasible exchanges in the system. Up to a certain point this results in a better download time ratio for sharing users, as the fraction of the total system capacity devoted to exchanges tends to increase. The improvement levels off and even decreases as the maximum number of outstanding requests increases. This can be explained by the increased competition between sharing users, that seems to reduce their relative benefit, although the reduction does not

appear to be significant.

We must note that since we do not explicitly model idle peers that have no outstanding requests, this also provides an indirect measure of the effect of idle users on system performance. Idle users do not participate in exchanges and therefore do not discriminate between sharing and non-sharing peers, dampening the effect of exchanges on relative performance.

The effect of the number of categories per peer is also significant, as it generally increases the probability of locating a feasible exchange. If the number of maximum outstanding requests is small, the effect appears to be reversed, with more categories per peer giving a slightly smaller relative benefit to sharing peers.

All of the previous simulations assumed a fixed fraction (50%) of peers were good citizens and shared. Figure 12 investigates the effect of frequency of uncooperative behavior on mean download times, to see whether incentives to share continue even if the vast majority of peers do not cooperate (or,

contrarily, if almost everyone cooperates). The measurements show that the gap in mean download times persists, regardless of the fraction of non-sharing nodes. The explanation for this is straightforward. We use the “no-exchange” case as a baseline, i.e. mean download time in a system in which every transfer is granted and no preference is given to sharers. When almost everyone is sharing, then sharers get the same performance as no-exchange (sharers rarely get an advantage from sharing), however, the non-sharers get a large penalty. On the other hand, when almost everyone is non-sharing, they rarely compete with a sharer, so the non-sharers receive the same performance as “no-exchange”. However, the infrequent sharer gets a big reward, because they are almost always able to preempt other transfers and get immediate service.

V. DISCUSSION

For the purpose of this study we have focused on a rather simplistic simulation scenario, and a specific file-sharing model. We discuss some of the limitations of our analysis and how the mechanisms proposed could be improved further.

In terms of simulation, the basic assumptions (e.g. overprovisioned core network, asymmetric bandwidth, zipf-like popularity) seem to agree with real-world measurements. Many of the other characteristics of the current model tend to err on the conservative side. Firstly, we assume that a peer cannot serve an object unless it has been fully received. In reality, many peer-to-peer systems (for example, eMule[15]) do serve “chunks” of incomplete objects. If this is incorporated in the model, the opportunity for exchanges is likely to increase further. In fact, this form of exchange is implemented in the Bittorrent system[19].

Secondly, we have assumed that transfer slots are fixed, regardless of the type of transfer. This means that exchanges cannot use more capacity than the standard transfer slot, although peers would clearly have interest in doing so. We also assume that a peer can only have one registered request on a given peer for a given object. Thus, if multiple exchanges are possible, whether pairwise or with different values of N , only one can be chosen.

Thirdly, in our simulation all peers have very similar characteristics, ignoring the widespread heterogeneity observed in real-world systems. For example, the existence of “super-peers” (e.g., peers with substantially better network capacity or storage) is likely to have a positive effect on exchange mechanisms, especially as a way to stimulate the deployment of exchange-capable clients.

Finally, we have ignored the complexity issues of communicating request tree information. The cost of communicating the full request tree may be prohibitive for peers with a large number of incoming requests and peers close to them in the request graph. If the request tree is updated incrementally, this is likely to introduce some latency in the search process which is not reflected in the waiting times of exchange transfers in the current simulation model. However, there are ways of reducing this cost. In particular, we can use a set of Bloom filters[23] to

represent the set of peers in the request tree², and ignore the detailed structure of the request tree and the objects associated with each edge. This is likely to offer significant savings especially considering the size of object and file identifiers in modern file sharing systems and the likelihood of the same peers. The main difference in the process of setting up an exchange is that the initiator does not have access to the full subgraph, and can only determine that a cycle exists, but cannot identify all the members of the exchange. The initiator must depend on next-hop lookups at each node instead of source-routing the request token around the ring, and there is a non-zero chance of false positives due to the probabilistic nature of Bloom filters. The space savings of this scheme are likely to be important for peers with a large number of incoming requests, and the peers close to them in the request graph. The details of this scheme (e.g. how to adapt, how to use hybrids consisting of both Bloom filters and complete request trees, how to protect against cheating, etc.) are subject to future work.

It is hard to speculate on how the incentives provided by exchanges would affect peer behavior. One direction for future work is therefore to determine how the proposed exchange mechanisms interact with replication mechanisms. In the current model, peers only store objects they are directly interested in. In an exchange system, users have an incentive to replicate popular objects that are in demand, as this is likely to increase their chances of participating in exchanges which, being prioritized, would give them a performance improvement. In essence, popular objects take the role of currency in exchange economies as they are easily exchangeable for other goods.³

There are three main limitations in the exchange approach as presented in this paper. First, peers are assumed to be equally interested in all of their requests. It would be useful to investigate non-uniform utility models, as many existing file-sharing systems already allow users to express their preference in terms of “low”, “medium” or “high” priority. This kind of information could be used to negotiate asymmetric exchanges, based on some form of bargaining or auction.

Second, peers can leverage their ability to participate in exchanges only when they have pending requests themselves. It would be desirable to provide a stronger incentive for users to actually stay connected even if they cannot immediately participate in some exchange.

Finally, we have only considered peers for which downloading increases utility and uploading either does not affect or reduces utility. The incentive structure is different in peer-to-peer content distribution systems: peers increase utility by pushing content out, and cooperating peers could help by downloading and re-distributing content on behalf of others. It would be interesting to see how exchange mechanisms can

²We require a different Bloom filter for each level in the request tree so that peers can trim the request tree by one level when they initiate a new request. Further, we need a distinct set of Bloom filters for each entry at the top level of the incoming request queue, so that a peer can reconstruct the next hop in the exchange ring when a ring initiation token comes down.

³We do not expect users to manually replicate objects or to patch their file-sharing software for that purpose. Rather, it is likely that developers of file-sharing clients will seize the opportunity to build software that is expected to perform better.

be adapted for this purpose, and in what form.

VI. SUMMARY AND CONCLUDING REMARKS

We have presented an exchange-based approach for providing incentives for cooperation in peer-to-peer file-sharing networks. Our approach is decentralized, and is considerably simpler than systems that provide system-wide forms of credit or cash. The basic idea is that peers give higher service priority to requests from a set of peers that can (transitively) provide a simultaneous, symmetric service in return. We describe methods for discovering sets of feasible n -way exchanges, and the methods for regulating transfers to provide incentives to share resources. We have also discussed how to guard these mechanisms against attacks by users wishing to exploit them to increase their own performance.

We have used simulation to analyze the mechanisms and determine their effect on performance. Our results show that exchange mechanisms offer a significant performance advantage to cooperating users, in terms of object download times. The performance advantage is more pronounced when the system gets more loaded, and when object popularity leans more towards a zipf-like distribution. Our results also show that higher-order exchanges offer a noticeable improvement (with improvements significantly diminishing with $n > 5$, if used together with pairwise exchanges. Thus, the proposed approach provides a strong incentive for users to share resources.

REFERENCES

- [1] E. Adar and B. A. Huberman, "Free riding on gnutella," *First Monday*, vol. 5, no. 10, October 2000.
- [2] "Kazaa Media Desktop," <http://www.kazaa.com/>.
- [3] "Kazaa Hack 2.1," <http://kazaahack.250x.com/>.
- [4] "Hack kazaa participation level - the easy answer," <http://www.davesplanet.net/kazaa>.
- [5] "MojonNation," <http://www.mojonation.net/Mojonation.html>.
- [6] John Ioannidis, Sotiris Ioannidis, Angelos Keromytis, and Vassilis Prevelakis, "Fileteller: Paying and Getting Paid for File Storage," in *Proceedings of the Sixth International Conference on Financial Cryptography*, March 2002.
- [7] Vivek Vishnumurthy, Sangeeth Chandrakumar, and Emin Gun Sirer, "KARMA: A secure economic framework for P2P resource sharing," in *Proceedings of the 1st Workshop on Economics of Peer-to-Peer Systems*, June 2003.
- [8] Sepandar D. Kamvar, Mario T. Schlosser, and Hector Garcia-Molina, "EigenRep: Reputation management in P2P networks," in *Proceedings of the 12th International World Wide Web Conference (WWW 2003)*, May 2003.
- [9] Ben Y. Zhao, John Kubiawicz, and Anthony D. Joseph, "Tapestry: An infrastructure for fault-tolerant wide-area location and routing," Tech. Rep. UCB/CSD-01-1141, April 2001.
- [10] Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, and Hari Balakrishnan, "Chord: A scalable peer-to-peer lookup service for internet applications," in *Proceedings of the ACM SIGCOMM '01 Conference*, San Diego, California, August 2001.
- [11] Sylvia Ratnasamy, Paul Francis, Mark Handley, Richard Karp, and Scott Shenker, "A scalable content addressable network," in *Proceedings of ACM SIGCOMM 2001*, August 2001.
- [12] John R. Douceur, "The sybil attack," in *Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS)*, Cambridge, MA, March 2002.
- [13] M. H. Goldhaber, "The attention economy and the net," *First Monday*, vol. 2, no. 4, April 1997.
- [14] Andrew Odlyzko, "The case against micropayments," in *Financial Cryptography 2003*, J. Camp and R. Wright, eds, *Lecture Notes in Computer Science*, Springer, 2003.
- [15] "The eMule Project," <http://www.emule-project.net/>.
- [16] "Official eMule-Board," <http://www.emule-project.net/board/index.php>.
- [17] Brent Chun, Yun Fu, and Amin Vahdat, "Bootstrapping a distributed computational economy with peer-to-peer bartering," in *Proceedings of the 1st Workshop on Economics of Peer-to-Peer Systems*, June 2003.
- [18] Larry Peterson, Tom Anderson, David Culler, and Timothy Roscoe, "A Blueprint for Introducing Disruptive Technology into the Internet," in *Proceedings of the 1st ACM Workshop on Hot Topics in Networks (HotNets-I)*, October 2002.
- [19] Bram Cohen, "Incentives build robustness in bittorrent," in *Proceedings of the 1st Workshop on Economics of Peer-to-Peer Systems*, June 2003.
- [20] Eric Friedman and Paul Resnick, "The social cost of cheap pseudonyms," *Journal of Economics and Management Strategy*, vol. 10, no. 2, pp. 173–199, 2001.
- [21] Mario T. Schlosser, Tyson E. Condie, and Sepandar D. Kamvar, "Simulating a P2P file-sharing network," in *First Workshop on Semantics in P2P and Grid Computing*, December 2002.
- [22] Jacky Chu, Kevin Labonte, and Brian Neil Levine, "Availability and locality measurements of peer-to-peer file systems," in *Proceedings of ITCOM: Scalability and Traffic Control in IP Networks II Conferences. Proceedings of SPIE Vol. No. 4868*, July 2002.
- [23] Burton H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970.